

广告桥接系统 · 开发 Work Plan

对应《广告桥接 Agent 架构设计》v0.3。前后端统一使用 TypeScript，工程骨架直接复用 Toonflow-app 现有代码，模型调用对齐 ads-gen-agent-main 已验证的选型（Claude / Seedance 2.0 / Whisper）。

状态 草案，待评审 日期 2026-07-23 前提 Phase 1 · 仅 Episode 尾插

01 - 概述

怎么定的这份计划

这不是从零设计一套新工程——Toonflow-app 已经有一整套「Express + Socket.IO + SQLite + 统一模型供应商层」的骨架，且已经内置了 Seedance 2.0 的视频供应商适配；ads-gen-agent-main 已经跑通了「多模态分析 → 创意生成 → 视频渲染」这条链路要用的具体模型。这份计划的核心工作是把架构设计文档里的 Agent 按 Toonflow 的工程约定搭出来，模型选型直接照抄 ads-gen-agent 已验证的配置，而不是重新选型。

阅读前提：本仓库里 Toonflow 前端只有编译产物（data/web），没有源码。从产物特征（Vue 3 编译期 `_ctx`、渲染函数签名、VITE_DEV 环境变量）判断其技术栈是 **Vue 3 + TypeScript + Vite**，前端相关任务按此假设排期，若实际前端仓库技术栈不同需要调整 M5。

02 - 技术栈

整体选型

后端语言/运行时

TypeScript + Node.js

前端语言/框架

TypeScript + Vue 3 + Vite

复用 Toonflow 的 Express 5 + Socket.IO + SQLite(Knex) 骨架

对齐 Toonflow 现有前端技术栈（从编译产物推断）

AGENT 编排

Vercel AI SDK (ai 包)
工具调用式编排，与 Toonflow 现有 `u.Ai.Text().stream()` 用法一致

文本/多模态模型

Claude · claude-opus-4-8
与 ads-gen-agent 相同 `base_url`（`napi.moretoken.ai`），复用 `@ai-sdk/anthropic`

视频生成

Seedance 2.0
Toonflow 已有 `atlascloud.ts/volcengine.ts/toonflow.ts` 供应商内置支持，无需新增供应商代码

语音转写

Whisper（本地，ONNX）
复用 Toonflow 已有依赖 `@huggingface/transformers`，避免另起 Python 进程

哪些是照搬，哪些要新写

模块	来源	说明
Express + Socket.IO 启动、JWT 鉴权、REST 自动路由	Toonflow	直接复用 <code>src/app.ts/src/core.ts/src/router</code> 骨架，新业务路由按同样的文件路径约定自册
统一模型供应商层 <code>u.Ai.Text/Image/Video/Audio</code>	Toonflow	原样复用，含参考图 <code>base64</code> 直传能力（不需额外对象存储上传素材）
Seedance 2.0 视频供应商适配	Toonflow	<code>atlascloud.ts</code> 的 <code>seedanceImageToVideo</code> 正好支持多参考输入，匹配「用已确认分镜草案作参考帧」需求
Socket 会话结构、聊天消息流式协议	Toonflow	照抄 <code>scriptAgent.ts/productionAgent.ts</code> 连接/鉴权/chat/stop 结构与 <code>ResTool</code> 协议
Agent Memory（RAG + 摘要）	Toonflow	直接复用 <code>src/utils/agent/memory.ts</code> 按新 Agent 的 <code>isolationKey</code> 接入即可

模块	来源	说明
Episode 抽帧 + 尾部密集抽帧 + 转写 + 多模态分析思路	ads-gen-agent	移植 DramaAgent 的分析逻辑到 TS，输出结构化 JSON
广告素材抽帧/转写/文字素材处理思路	ads-gen-agent	移植 AdAgent 逻辑到 TS，三种输入类型平支持
创意方案生成与六维评分思路	ads-gen-agent	移植 bridge_agent.py/evaluator.py 「生成→评分→修改」循环结构，评分维度复用
Playable 资产装配（模板/外部 H5 包两种模式）	ads-gen-agent	移植 build_playable.py 到 TS——纯文装配 + ffmpeg 拼接逻辑，无 Python 专有依赖，可直接改写
SessionAgent / StoryboardAgent / AdLibraryAgent / DirectorAgent / 三个执行 Agent / 四个 Evaluator / SupervisorAgent / Assembler 的 TS 实现	新增	按架构设计文档从零实现，工程写法照抄 Toonflow src/agents/* 的既有模式
方案候选卡片 / 分镜草案 cut 卡片 / 内容候选卡片的前端组件	新增	需要访问实际前端仓库源码后落地，本仓库编译产物

每个 Agent 具体调什么模型

AGENT / 用途	模型	调用方式
StoryboardAgent 多模态分析	claude-opus-4-8	u.A 为消
StoryboardAgent 语音转写	Whisper (本地 ONNX)	@hu 与 E
AdLibraryAgent 分析	claude-opus-4-8	u.A
SessionAgent 意图解析 + 调度	claude-opus-4-8	u.A 用编

AGENT / 用途	模型	调用方
DirectorAgent 创意方案	claude-opus-4-8	u.A Sess
PlanEvaluator / BridgeVideoEvaluator / PlayableEvaluator / OverlayEvaluator	claude-opus-4-8	u.A JSO
BridgeVideoAgent · Stage A 分镜草案	Toonflow 已配置的图片模型（选型不锁定）	u.A
BridgeVideoAgent · Stage B 成片渲染	Seedance 2.0 (doubao-seedance-2-0 / bytedance seedance-2.0)	u.A ima 认 c
SupervisorAgent 终 审	claude-opus-4-8	u.A

05 - 里程碑

建议的实现顺序

按依赖关系排序，不含具体日期（团队人力/排期未知）；每项标了相对工作量供参考。

M0 基建对齐

S

- 新建业务数据表：ab_episode / ab_ad / ab_creativePlan / ab_bridgeCut / ab_generatedSegment / ab_manifest
- 确认/配置 Claude 供应商（base_url、模型名与 ads-gen-agent 一致）
- 确认 Seedance 供应商在当前项目里已启用，跑通一次最小 u.Ai.Video() 调用
- 验证 @huggingface/transformers 本地 Whisper 转写的可用性与准确率

依赖 无 交付物 可跑通的最小 Text/Image/Video/Whisper 调用示例

M1 批处理层：StoryboardAgent + AdLibraryAgent

M

- `src/agents/storyboardAgent/index.ts`: 抽帧 + 转写 + 多模态分析 → 结构化 `episodeAnalysis`
- `src/agents/adLibraryAgent/index.ts`: 视频/图片/文字三态输入 → 结构化 `ads[]`, 含 `hasVisualAsset`
- REST 路由: 上传触发分析、查询分析状态

依赖 M0 交付物 上传一个 Episode + 广告素材, 能拿到结构化分析结果

M2 SessionAgent 调度层 + DirectorAgent 方案层

M

- Socket 命名空间 `/api/socket/sessionAgent` (照抄现有 Agent 命名空间结构), 内含确定性状态机 (`uploaded`→`plan_review`→`content_review`→`assembling`)
- DirectorAgent 创意方案生成 (多候选, 支持混合形式序列) + skill 文件, 内部调用 `PlanEvaluator`
- 方案候选卡片消息协议 + 针对单个方案的 `revise` 路由逻辑 (自由文字反馈才过 LLM 解析, 按钮类操作走确定性代码)

依赖 M1 交付物 能对话式产出/修改/确认创意方案, 尚不触发实际生成

M3 执行层: 三种形式

L

- BridgeVideoAgent: Stage A 分镜草案生成/逐张 `revise`, Stage B 成片渲染, `BridgeVideoEvaluator`
- PlayableAgent: 移植 `build_playable.py` 装配逻辑, `PlayableEvaluator`
- OverlayAgent: CTA 卡片生成/修改, `OverlayEvaluator`
- 三者统一由 SessionAgent 按已确认方案的形式序列调度

依赖 M2 交付物 三种形式各自能生成候选、接受局部反馈重生成

M4 监督与落地

S

- SupervisorAgent 落地前终审 + skill 文件
- Assembler: `ffmpeg` 拼接 Episode + 确认内容, 输出成片
- `manifest.json` 生成, 任务记录接入 `withTaskRecord()`

依赖 M3 交付物 端到端从确认内容到最终成片文件

M5 前端

L

- Episode 概览 + 广告列表展示
- 方案候选卡片 / 分镜草案逐张 review / 内容候选卡片三类 UI
- Socket 客户端接入，复用/参照 Toonflow 前端既有的聊天消息渲染体系（需要实际前端仓库源码）

依赖 可与 M2/M3 并行开发，需 Mock 数据 交付物 完整可操作的端到端 UI

M6 联调与验收

M

- 跑通一个真实 Episode + 广告素材的完整案例
- 验证 Evaluator 参考分展示、revise 循环、SupervisorAgent 拦截路径
- 成本/耗时实测，校准分镜草案与成片渲染阶段的真实耗时差异

依赖 M1–M5 交付物 可演示的端到端 Demo

06 – 关键技术决策

几个值得记录的选择

参考素材直接走 base64，不新增对象存储层

ads-gen-agent 用 GCS 给 Seedance 传参考素材 URL；Toonflow 现有供应商层已支持参考图/视频以 base64 直传（referenceList2imageBase642），可以跳过这一层，减少工作量。

Whisper 用本地 ONNX（transformers.js），不起 Python 子进程

前后端统一 TS 是硬性要求；Toonflow 已有 @huggingface/transformers 依赖跑本地 Embedding 模型，同一套运行时可以承载 Whisper，避免引入 Python 运行时依赖。

build_playable.py 移植为 TS，而不是保留 Python 子进程

该脚本本质是文件装配 + 模板注入 + ffmpeg 拼接，没有 Python 专有依赖，移植成本可控，符合全 TS 的要求。

07 - 待确认事项

这些需要你来定

- ? **前端源码访问**——本仓库只有 Toonflow 前端编译产物，M5 的组件级复用需要实际前端仓库源码才能精确规划。
- ? **Stage A 图片模型选型**——分镜草案用哪个具体图片模型，取决于项目当前实际配置的供应商，未锁定具体型号。
- ? **排期/人力**——本计划只给出里程碑依赖顺序和相对工作量（S/M/L），没有换算成日历时间，需要你提供团队规模才能排具体周期。